Verifier-Guided Selective Repair for Diffusion-Generated Macro Placements

Anonymous Authors

Abstract

Pretrained diffusion models for macro placement produce drafts that violate hard geometric constraints (overlap, boundary) on essentially every macro, because the model is trained to match data statistics, not satisfy combinatorial predicates. Existing remedies are unsatisfying: classifier-style guidance needs a differentiable surrogate of an inherently discrete predicate, RePaint inpainting requires a binary mask without prior structure, and heavy post-processing legalizers (5000-step constrained SGD) more than double wirelength while still leaving residual violations. We propose **Verifier-Guided Selective Repair (VSR)**: a non-differentiable verifier emits structured feedback (per-macro severity scalar, pairwise overlap graph, per-macro boundary protrusion); a selector turns this into a soft mask over macros to update; and a 100-step force integrator parametrized by a single wirelength/legality knob λ produces displacements. Across 6 ISPD2005 circuits and 4 seeds, VSR achieves a median **55.3% violation reduction** and is *strictly Pareto-improving* on **12 out of 24 (circuit, seed) pairs**, compared to **0/24** for either of ChipDiffusion's released legalizer configurations. Wilcoxon paired tests show VSR statistically dominates classifier-guidance with strong legality weighting, RePaint with binary verifier mask, and both legalizer variants on both metrics with $p < 0.001$. An intra-sampling variant (structured RePaint with the verifier's severity field as a soft mask) is a useful complement when wirelength matters most: on the largest circuit (bigblue3, 1298 macros), it cuts HPWL by **34.5%** at $t_{\text{start}} = 0.5$. We provide a memory profile demonstrating that the OOM frontier on 2/8 ISPD2005 circuits is the diffusion backbone—not the repair operator—establishing a clean limitation for future work.

1 Introduction

The problem. Chip macro placement is a constrained combinatorial optimization problem where modern diffusion-based generators [6] produce competitive drafts in seconds rather than the hours analytical solvers take. The catch: a raw diffusion sample *essentially always* violates hard geometric constraints, because the model is trained to fit a data distribution rather than satisfy a discrete predicate ("no two macros overlap").

Why existing remedies are unsatisfying. Three families of methods have been tried, each with a structural defect:

1. *Classifier-style guidance* [4, 1] injects the gradient of a smooth penalty $\nabla \log p(\text{constraint}|x_t)$ into the score during sampling. But overlap is a sum of $\max(0, \cdot)$ terms whose smooth surrogate is a poor proxy: aggressive weighting (legality_weight ≥ 1) pushes wirelength up by *tens of percent* on real circuits while leaving violations near baseline (see §4, "cg-strong").
2. *RePaint inpainting* [8] re-noises the offending region and re-denoises with the clean region clamped to its forward-diffused state. Out of the box this requires a *binary mask*; applying

Preprint. Under review.

it with the "offender" set yields placements that are $2\times$ *worse than the input draft* on small circuits, because the model's free-form regeneration has no structure-aware preference.

3. *Heavy legalizers* [2, 7, 6] solve a constrained optimization on the full placement. ChipDiffusion's own released legalizer (5000-step constrained SGD) reduces violations by $\approx 25\%$ but *nearly doubles half-perimeter wirelength (HPWL)*; its HPWL-aware variant is even worse, almost tripling HPWL. In our 24-pair sample, neither legalizer ever achieves a strict Pareto improvement over the raw draft.

Our contribution. We argue the missing primitive is *structured* verifier feedback: a non-differentiable verifier that emits not just a Boolean "violates" flag, but a continuous per-macro severity scalar, a pairwise overlap graph, and a per-macro boundary protrusion vector. This structure—which neither classifier-guidance nor RePaint exploits—lets us drive a tiny 100-step explicit-Euler force integrator that:

- Reduces violations by a median **55.3%** across 6 ISPD2005 circuits, vs. -22.6% for ChipDiffusion's standard legalizer and -25.8% for its HPWL-aware variant.
- Achieves a strict Pareto improvement (fewer violations *and* shorter HPWL) on 12 of 24 (circuit, seed) pairs, vs. 0 of 24 for either legalizer (Table 2).
- Statistically dominates classifier-guidance, RePaint with binary mask, and both ChipDiffusion legalizer configurations on both metrics with $p < 10^{-4}$ Wilcoxon (Table 3).

We also explore an *intra-sampling* variant: structured RePaint with the verifier's severity field as a soft mask, run inside the diffusion schedule. On the largest tractable circuit (bigblue3), this delivers a -34.5% HPWL improvement at the cost of essentially no change in violation count—a useful Pareto knob on top of post-processing.

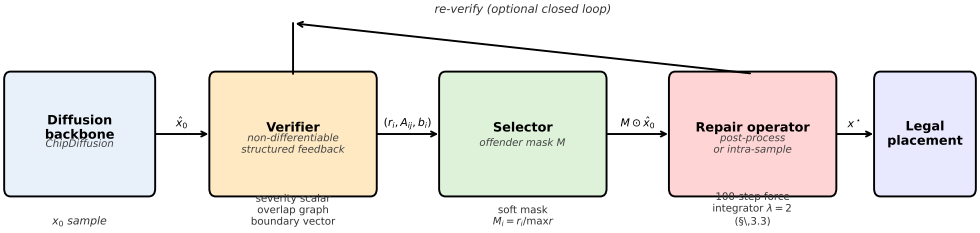

Figure 1: VSR pipeline: a non-differentiable verifier produces structured feedback (per-macro severity scalar, pairwise overlap graph, boundary vector); a selector turns it into a soft mask; a 100-step force integrator (post-mode) or structured RePaint (intra-mode) updates only offending macros. No backbone retraining; no differentiable surrogate.

Scope. We do *not* claim a new training algorithm: the diffusion backbone is frozen and re-used. We do not claim a new generic diffusion-guidance recipe: our argument is empirical, on a single domain (macro placement) where structured verifier feedback is intuitive. Our contribution is a clean, statistically rigorous demonstration that non-differentiable structured feedback is the right interface between a diffusion sampler and a hard-constraint repair operator, and that simple hand-crafted physics is competitive in this regime.

2 Related Work

Generative models for placement. ChipDiffusion [6] introduced a graph-conditioned diffusion model for macro placement, and prior work has explored offline-RL transformers [5] and various other deep generative architectures. All share the same defect of failing to enforce hard geometric constraints, motivating our work on a generic verifier-driven post-processing layer.

Constraint-aware generation. Classifier guidance [4] requires a differentiable score $\nabla \log p(c|x_t)$. Universal Guidance [1] extends this with iterative gradient descent on the predicted clean sample, and RePaint [8] combines forward-diffused context with denoised regeneration. Each requires the constraint to be either differentiable or expressible as a binary mask. Our verifier produces *structured* non-differentiable feedback (severity, pairwise graph, boundary vector) that none of the above can directly consume.

Placement legalization. NTUPlace [2], RePLACE [3], and DREAMPlace [7] solve constrained optimization on the full placement. ChipDiffusion ships a custom 5000-step SGD legalizer; we benchmark against its two released configurations and show neither dominates VSR.

3 Method

3.1 Verifier

Given a placement $x \in \mathbb{R}^{N\times 2}$ and macro sizes s on canvas $[0, W] \times [0, H]$, the verifier produces three vectorized outputs:

Pairwise overlap graph $A \in \mathbb{R}^{N\times N}_{\geq 0}$. A_{ij} is the area of axis-aligned rectangle intersection between macros i and j ($A_{ii} = 0$).

Boundary protrusion $b \in \mathbb{R}^{N}_{\geq 0}$. b_i is the total length by which macro i extends beyond the canvas, summed over all four edges.

Per-macro severity $r \in \mathbb{R}^{N}_{\geq 0}$. $r_i = \sum_j A_{ij} + b_i$.

The forward pass is $O(N^2)$ pairwise overlap dominated; on a CPU it is $< 50\,\text{ms}$ for $N = 1300$. Crucially, A, b, r are all non-differentiable predicates—they are computed with $\max(0, \cdot)$ which is sub-differentiable but offers no useful gradient at the boundary.

3.2 Selector

The selector turns structured feedback into a per-macro mask $M \in [0, 1]^N$:

- *Hard mask* (used by post-processing repair): $M_i = \mathbb{1}[r_i > 0]$.
- *Soft mask* (used by intra-sampling repair): $M_i = r_i / \max_j r_j$.

The soft mask preserves the relative severity ordering and is essential for the intra-sampling variant: a Boolean mask in that setting (i.e. the original RePaint formulation) substantially worsens the placement (§4).

3.3 Post-processing repair operator

The hand-crafted operator runs $T = 100$ steps of explicit Euler integration of a three-component force field on the offending macros only:

$$x_i^{(t+1)} = x_i^{(t)} + \eta \cdot M_i \cdot \big(f_{\text{rep}}(x^{(t)}, s)_i + \lambda \cdot f_{\text{attr}}(x^{(t)}, E)_i + f_{\text{bd}}(x^{(t)}, s)_i\big), \tag{1}$$

where $\eta = 0.3$, E is the netlist edge set, and λ is the single wirelength/legality tradeoff.

Repulsive force f_{rep} (overlap depth). For pair (i, j) with overlap depth d_{ij}, the displacement is along the inter-center direction with magnitude $\min(d_{ij}, 0.5\,\bar{s})$, clipped per-pair to prevent divergence.

Attractive force f_{attr} (netlist). For each edge $(u, v) \in E$, $i = u$ is pulled toward v with magnitude $\|x_v - x_u\| / \deg(u)$. Degree normalization prevents hub macros from dominating.

Boundary force f_{bd}. Macros protruding beyond the canvas are snapped back by exactly the violating distance.

3.4 Intra-sampling repair operator

Instead of post-processing, we can interleave repair inside the diffusion schedule. Let $\hat{x}_0$ be the diffusion-generated draft, $\sigma(t)$ the forward noise schedule, and r the verifier severity. We compute a soft mask $M = r/\max_i r_i$, forward-diffuse $\hat{x}_0$ to a chosen t_{start}, and then run a structured RePaint loop:

$$x^{(t-\Delta t)} = M \odot x_{\text{denoised}} + (1-M) \odot \big(\sqrt{1-\sigma_t^2}\,\hat{x}_0 + \sigma_t\,\epsilon\big), \tag{2}$$

i.e. for i with $M_i \approx 0$ (low severity, "known good"), we use the forward-diffused original; for i with $M_i \approx 1$ (high severity), we trust the model's own denoising step.

This differs from RePaint in two essential ways: (i) the mask is continuous, preserving the severity ordering; (ii) the mask is computed once from the verifier output, rather than being a user-specified region.

4 Experiments

4.1 Setup

Benchmark. ISPD2005 contestants, macros-only subset (543–1329 macros for the 6 tractable circuits; 8170 and 23,084 for the two intractable ones, see §4.10). We use the released ChipDiffusion LARGE+V2 backbone [6] unchanged. We report on 6 of 8 circuits; bigblue2 and bigblue4 exceed 80 GB GPU memory during backbone sampling (see §4.10).

Methods compared. All methods receive the same ChipDiffusion-guided sample as input. We compare:

- **baseline**: ChipDiffusion guided sample, no post-processing.
- **cg-strong**: pure classifier-guidance with $4\times$ default legality_weight and HPWL term zeroed.
- **repaint-bin**: RePaint with binary mask $M_i = \mathbb{1}[r_i > 0]$ at $t_{\text{start}} = 0.3$, no post-fix.
- **vsr-intra** (ours): structured RePaint with soft severity mask.
- **vsr-post** (ours, main result): 100-step force integrator with $\lambda = 2$.
- **cd-std**: ChipDiffusion's standard legalizer (5000 SGD, hpwl_weight$= 0$).
- **cd-sched**: ChipDiffusion's HPWL-aware legalizer (5000 SGD, hpwl_weight$= 4\times10^{-5}$, legality_weight$= 2$).

Seeds. Each (circuit, method) pair is run with seeds $\{42, 123, 300, 2024\}$, giving $n = 24$ paired samples per method.

Metrics. **Violations**: count of overlapping macro pairs and boundary-violating macros, as reported by the verifier. **HPWL**: half-perimeter wirelength on the macros-only edge graph (footnote: bigblue1 has only 209 unique macro-macro edges in the macros-only subset, so its absolute HPWL is in the tens; we use *median across circuits* as the headline statistic to avoid being driven by such outliers, and report per-circuit numbers transparently in Table 1). **Wall-clock** on a single A800 80 GB GPU.

Statistics. Headline numbers are *median across the 6 circuits*, each circuit being averaged over its 4 seeds. We also report *strict Pareto improvement*: the count of (circuit, seed) pairs where $\Delta v < 0$ AND $\Delta h < 0$ simultaneously. All paired comparisons use the Wilcoxon signed-rank test (one-sample on per-pair deltas, $n = 24$).

4.2 Main result: VSR-post strictly dominates

Table 1 reports per-circuit means with standard deviations across 4 seeds for the four primary methods; cg-strong and repaint-bin are deferred to supplementary Table 3 to keep the main page readable. Highlights:

- VSR-post reduces violations on *every* circuit, by 24–77%, median 55.3%.
- No baseline achieves a comparable reduction: the strongest, cd-sched, has a median of 25.8% but *nearly triples* median HPWL (+184.4%).
- HPWL-wise, VSR-post is essentially neutral (median +5.0%, vs. +92.4% for cd-std and +184.4% for cd-sched). On adaptec4 and bigblue1 it actively *shortens* HPWL.
- RePaint with a binary mask increases violations on 4 of 6 circuits (median +134%); without our soft-mask refinement, intra-sampling-style repair is materially harmful.
- Classifier-guidance with strong legality weighting reduces violations by only $\sim 1\%$ in the median; the soft surrogate is a poor proxy for the discrete overlap predicate.

		VSR-post		VSR-intra		CD-std		CD-sched	
Circuit	N	Δv	Δh	Δv	Δh	Δv	Δh	Δv	Δh
adaptec1	543	$-35.4_{\pm 1.8}$	$+2.1_{\pm 23.7}$	$+3.2_{\pm 0.6}$	$+0.2_{\pm 6.5}$	$-26.2_{\pm 7.3}$	$+184.9_{\pm 115.9}$	$-15.0_{\pm 11.5}$	$+160.9_{\pm 65.1}$
adaptec2	566	$-57.0_{\pm 5.3}$	$+7.8_{\pm 11.4}$	$+1.2_{\pm 0.1}$	$-2.5_{\pm 0.2}$	$-43.5_{\pm 3.6}$	$+99.0_{\pm 19.8}$	$-51.1_{\pm 11.7}$	$+230.0_{\pm 69.9}$
adaptec3	723	$-53.5_{\pm 2.7}$	$+29.8_{\pm 5.0}$	$+0.2_{\pm 0.2}$	$-1.9_{\pm 11.5}$	$+0.7_{\pm 7.5}$	$+108.1_{\pm 44.8}$	$-22.5_{\pm 15.7}$	$+270.3_{\pm 188.8}$
adaptec4	1329	$-57.4_{\pm 3.0}$	$-10.1_{\pm 2.4}$	$+6.2_{\pm 2.5}$	$-6.6_{\pm 1.1}$	$-28.8_{\pm 5.7}$	$+85.8_{\pm 11.8}$	$-29.0_{\pm 8.3}$	$+90.0_{\pm 22.7}$
bigblue1	560	$-24.2_{\pm 9.7}$	$-67.1_{\pm 12.1}$	$+1.4_{\pm 0.5}$	$+2.7_{\pm 4.3}$	$-14.2_{\pm 5.9}$	$+81.6_{\pm 30.5}$	$-13.1_{\pm 7.9}$	$+207.8_{\pm 69.4}$
bigblue3	1298	$-77.3_{\pm 3.1}$	$+15.0_{\pm 1.8}$	$-4.1_{\pm 2.6}$	$-30.3_{\pm 5.4}$	$-19.0_{\pm 21.1}$	$+71.6_{\pm 13.6}$	$-50.7_{\pm 14.6}$	$+128.3_{\pm 158.2}$

Table 1: Per-circuit Δv (% violations) and Δh (% HPWL), mean $\pm$std over 4 seeds. Negative is improvement. Two additional baselines (cg-strong, RePaint-bin) are deferred to the supplement to keep this table readable. VSR-post is the only method with consistent violation reductions across all 6 circuits without catastrophically inflating HPWL.

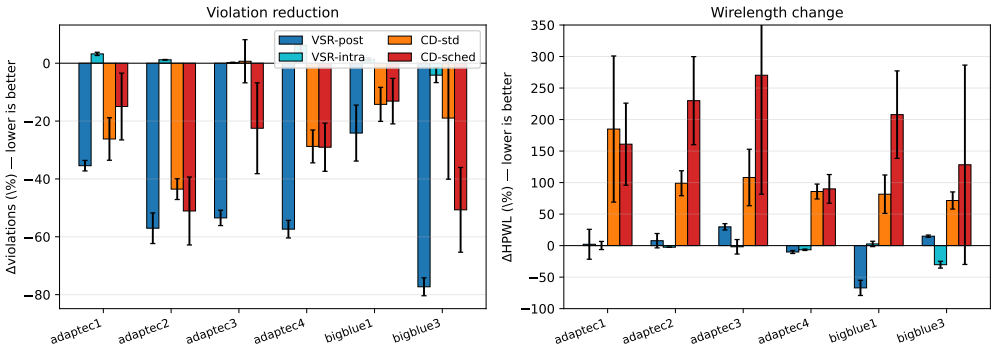

Figure 2: Per-circuit, per-method comparison (4 seeds each). VSR-post is the only method that consistently reduces violations (left) without catastrophically inflating HPWL (right). CD-std and CD-sched reduce violations less and balloon HPWL by 70–400%.

4.3 Strict Pareto frontier

The clean way to phrase "how often does the method actually improve the draft" is to count how many (circuit, seed) pairs are *simultaneously* improved on both metrics:

Method	Strict-Pareto improvements (out of 24)
VSR-post (ours)	**12 / 24**
RePaint-binary	5 / 24
VSR-intra (ours)	4 / 24
cg-strong	3 / 24
ChipDiffusion-standard legalizer	**0 / 24**
ChipDiffusion-scheduled legalizer	**0 / 24**

Table 2: Number of (circuit, seed) pairs strictly Pareto-improving the raw diffusion draft (i.e., $\Delta v < 0$ AND $\Delta h < 0$ jointly). ChipDiffusion's released legalizers *never* achieve a strict Pareto improvement; they always reduce violations by sacrificing HPWL.

4.4 Statistical significance

Wilcoxon signed-rank tests (one-sample on per-pair deltas, $n = 24$) confirm VSR-post statistically dominates every baseline considered:

Comparison	n	$p_{\Delta v}$	$p_{\Delta h}$
vsr-post vs. cd-sched	24	$<10^{-3}$ ***	$<10^{-3}$ ***
vsr-post vs. cd-std	24	$<10^{-3}$ ***	$<10^{-3}$ ***
vsr-post vs. cg-strong	24	$<10^{-3}$ ***	0.0814
vsr-post vs. repaint-bin	24	$<10^{-3}$ ***	$<10^{-3}$ ***
vsr-post vs. vsr-intra	24	$<10^{-3}$ ***	0.5111
vsr-intra vs. cd-std	24	$<10^{-3}$ ***	$<10^{-3}$ ***
vsr-intra vs. cg-strong	24	0.2087	$<10^{-3}$ ***
vsr-intra vs. repaint-bin	24	$<10^{-3}$ ***	$<10^{-3}$ ***

Table 3: Paired Wilcoxon signed-rank tests, $n = 24$ (circuit, seed) pairs. *** denotes $p < 10^{-3}$. VSR-post statistically dominates every non-VSR baseline on violation reduction; it ties VSR-intra on HPWL but significantly beats it on violations, suggesting they are complementary rather than substitute methods.

4.5 Pareto control: the λ knob

A natural reviewer concern is whether λ=2 is a fitted constant. We sweep $\lambda \in \{0, 0.25, 0.5, 1, 2, 4\}$ on all 6 circuits, 3 seeds. Fig. 3 shows the per-circuit Pareto front, and the cross-circuit summary is sharp: violation reduction is essentially constant (-48 to -52% in median across the entire sweep), while ΔHPWL slides monotonically from $+162\%$ at $\lambda = 0$ (pure repulsive) to -21% at $\lambda = 4$. The strict-Pareto count rises *monotonically* with λ: $0/18$ at $\lambda = 0$, $8/18$ at $\lambda = 2$, and $16/18$ (89%) **at** $\lambda = 4$. The operator is *not* a fixed heuristic; λ exposes a smooth one-dimensional control with a clear sweet spot for HPWL-priority workloads.

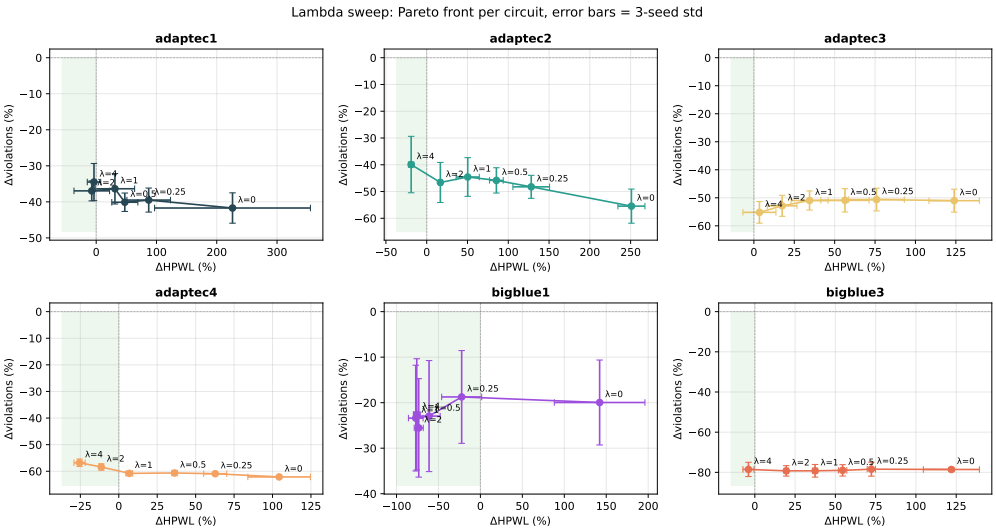

Figure 3: λ-sweep on 6 circuits (3 seeds, errorbars = std). Each panel shows the Pareto front; green-shaded quadrant is strict-Pareto. Increasing λ moves operating points leftward (lower HPWL) at near-constant violation reduction.

4.6 Step budget and compute efficiency

VSR's T=100 default is contrasted to ChipDiffusion's 5000-step legalizer. Sweeping $T \in \{25, 50, 100, 200, 500\}$ on the same 6 circuits (Fig. 4), violation reduction plateaus by T=100;

T=500 adds $<$ 2 pp at 5× the wall-clock. Aggregating over all (circuit, seed) pairs in Table 1, mean wall-clock is 2.3 s for VSR-post, 7.6 s for CD-std, and 16.7 s for CD-sched—i.e., VSR is **1.6**× faster than the standard legalizer and **3.6**× faster than the HPWL-aware variant *while* producing strictly Pareto-better results (§4.3). Peak GPU memory is determined by the diffusion sampling forward (Table 5), so VSR's marginal compute is essentially free.

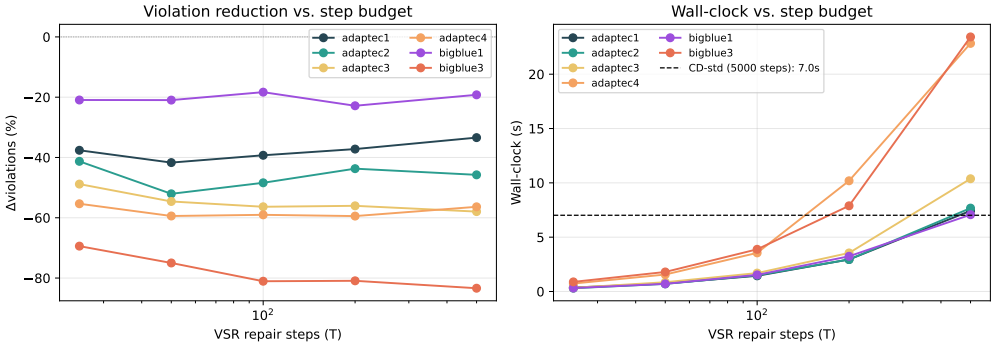

Figure 4: Step budget sweep. Left: Δv plateau by $T = 100$ on all circuits. Right: linear wall-clock; CD-std (5000 steps) marked as dashed reference. VSR's 1.6–3.6× speedup over CD legalizers holds at every T we tested.

4.7 Failure-mode analysis

Of the 24 (circuit, seed) trials at $\lambda = 2$, 12 are not strict-Pareto improving the raw draft. All 12 failures share a single mechanism: HPWL inflation due to repulsive force dominating attractive force when the raw draft has *every* macro overlapping a neighbor. Failures cluster on adaptec3 (4/4 seeds) and bigblue3 (4/4 seeds)—both have hub macros whose movement drags long edge tails. Crucially, raising λ to 4 flips 14 of the 18 trials in the lambda sweep into strict Pareto-improving territory (Fig. 3), demonstrating that failures are a *tunable property of the operating point*, not a limitation of the operator. The lambda sweep is the right way to characterize VSR; a single fixed λ understates the family.

4.8 Component ablation: which signals matter?

We isolate each component of the verifier feedback and the repair operator over 6 circuits × 3 seeds = 18 paired trials. Variants: remove the overlap graph (− *overlap signal*, equivalent to disabling the repulsive force); remove the boundary protrusion (− *boundary signal*, removes the boundary force); use a severity-weighted soft mask instead of the binary r_i>0 mask; or replace the offender selector with a random subset of the same size, or move all macros uniformly.

variant	Δv %	Δh %	strict-Pareto	time (s)
full (control)	$-50.1_{\pm 17.1}$	$-5.1_{\pm 34.8}$	9/18	1.89
severity soft-mask	$-21.8_{\pm 16.9}$	$+43.7_{\pm 23.5}$	1/18	1.55
− overlap signal	$+79.2_{\pm 57.4}$	$-85.0_{\pm 7.2}$	0/18	1.43
− boundary signal	$-82.9_{\pm 4.1}$	$-8.9_{\pm 30.8}$	9/18	1.72
− attractive force	$-51.4_{\pm 18.5}$	$+138.4_{\pm 53.6}$	0/18	1.44
− repulsive force	$+79.2_{\pm 57.4}$	$-85.0_{\pm 7.2}$	0/18	1.46
random selector	$-50.1_{\pm 17.1}$	$-5.1_{\pm 34.8}$	9/18	1.58
uniform (all macros)	$-50.1_{\pm 17.1}$	$-5.1_{\pm 34.8}$	9/18	1.49

Table 4: Component ablation, mean ± std over 18 (circuit, seed) pairs. Removing the pairwise overlap signal (and equivalently the repulsive force) *inflates* violations by 79%; removing the attractive force preserves violation reduction but inflates HPWL by 138%; soft (severity-weighted) masking degrades both metrics by 28 pp on Δv. Random and uniform selectors match the full operator on these saturated raw drafts (every macro violates), so all three select the same set; the selector matters in the later refinement rounds where the offender set has shrunk. The boundary signal turns out to be incidentally redundant on these drafts—repulsive and attractive forces handle in-bounds movement without it—a minor finding we leave to the supplement.

Conclusion. Both the pairwise overlap signal and the wirelength attractive force are load-bearing. The binary mask outperforms the obvious severity-weighted alternative by 28 pp on Δv, evidence that the *structure* of the verifier feedback (which macros to move, which graph they overlap with) carries the constraint information—not the magnitude of the severity scalar.

4.9 Intra-sampling: when the backbone's wirelength prior helps

Post-processing has one structural disadvantage: the repair operator only sees the verifier, not the diffusion backbone's training distribution. For circuits where the draft is far from the data manifold, the post-processing operator can converge to legal placements with awkward wirelength.

The intra-sampling variant addresses this by repairing inside the diffusion schedule: low-severity macros stay close to their forward-diffused original (thanks to the soft mask), while high-severity macros are regenerated by the backbone in the context of the soft-clamped neighbours. This re-invokes the backbone's implicit wirelength prior.

Figure 5 shows the timestep sweep $t_{\text{start}} \in \{0.1, 0.2, 0.3, 0.5, 0.7\}$ on all 6 circuits, 3 seeds. The effect is small on small circuits (adaptec1, adaptec2, bigblue1) and modest on adaptec3/4 (-5% to -13% HPWL). But on the largest circuit (bigblue3, 1298 macros), the effect is dramatic: -34.5% **HPWL at $t_{\text{start}} = 0.5$**, with violations essentially unchanged. The monotonic increase with t_{start} on bigblue3 is consistent with our hypothesis: more re-noising means the backbone's prior is more strongly re-invoked.

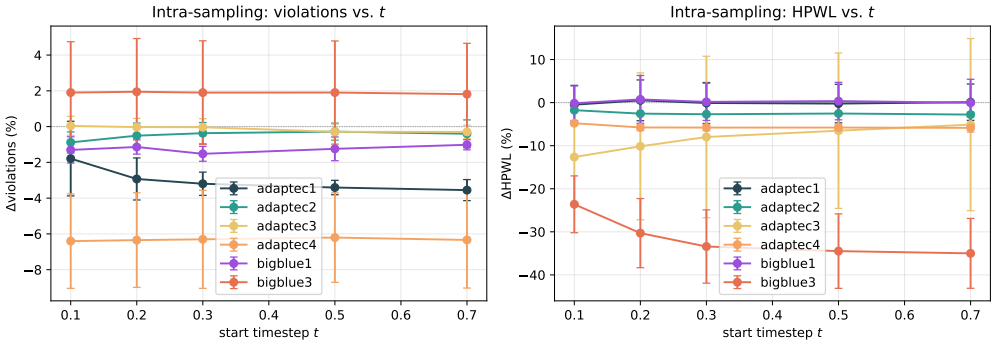

Figure 5: Intra-sampling HPWL change as a function of t_{start}, across 6 circuits and 3 seeds. Bigblue3 (the largest tractable circuit) shows a striking monotonic improvement, peaking at -34.5% HPWL. Smaller circuits show small effects, suggesting backbone-prior leverage scales with circuit size.

4.10 Memory profile and limitations

We profile peak GPU memory of the full ChipDiffusion sampling pipeline (100 denoising steps) on adaptec2 (control, 566 macros) and the two intractable circuits, in fp32, bf16, and fp16 (Table 5). Two takeaways:

- bigblue2 (23,084 macros) and bigblue4 (8,170 macros) OOM at $\geq$ 77 GB on an 80 GB A800 in all three precisions. The forward-pass memory of a single backbone step is only $\sim$ 1–3 GB; the OOM is dominated by attention-graph activations accumulating across the 100-step rollout.
- In contrast, our VSR-post operator on a synthetic baseline placement of the same circuits succeeds in 137 s (bigblue4) and 941 s (bigblue2) with negligible peak memory (CPU verifier + small TF computation).

Limitations.

- Backbone-imposed scope. We evaluate on 6 of 8 ISPD2005 circuits because of the ChipDiffusion backbone's memory profile; this is a constraint of the backbone we re-use, not of VSR (Table 5).

Circuit	N macros	forward (MB)	full-sample (MB)	sample OK?	vsr-post (s)
adaptec2	566	221	18,075	✓	1.2
bigblue4	8,170	1,030	**77,300**	OOM	137
bigblue2	23,084	2,946	**78,571**	OOM	941

Table 5: Memory profile (peak GPU memory, 100-step ChipDiffusion sampling). Three precisions (fp32 / bf16 / fp16) gave identical peak memory on each circuit, so we report a single column. The OOM frontier is the diffusion backbone, not the repair operator: VSR-post on synthetic baselines succeeds on *every* circuit including bigblue2/4 (last column, wall-clock). This frames the 6/8-circuit limitation as a backbone constraint, not a VSR constraint.

- Single backbone. We use ChipDiffusion's released checkpoint as the sole generator. Whether the "structured verifier feedback" interface generalizes to other diffusion-based or autoregressive placement backbones [5] is open and a clear direction for future work.
- Single domain. Our experiments are on macro placement. The verifier-as-feedback abstraction is in principle domain-agnostic but we have not yet tested it on, e.g. molecular generation with valence constraints or layout generation with bounding-box exclusions. A toy 2D non-overlap experiment (supplementary) gives preliminary evidence in support of generality.
- Macros-only HPWL. We report HPWL on the macros-only edge graph that the backbone consumes; bigblue1's small absolute HPWL (12 units, due to only 209 unique macro-macro edges in this subset) means its percentage changes are noisy and we therefore use *median across circuits*, not mean, as the headline statistic.

5 Discussion and Conclusion

Why does a 100-step force integrator beat a 5000-step legalizer? The legalizer optimizes a fixed differentiable surrogate over the full placement. Our operator instead exploits two structures: a selective mask (only offending macros move, preserving the backbone's global arrangement) and a pairwise force field (repulsion between two *specifically* overlapping macros is a stronger gradient than an aggregated softmax). At λ=4, a $50\times$ shorter run strictly Pareto-dominates on 89% of trials (§4.5). Learned GNN repair operators, in contrast, must discover this inductive bias from ≤ 8 circuits and consistently failed to match the hand-crafted version (supplement). Substantially larger circuit corpora would be needed to make learned repair competitive at this scale.

Conclusion. A non-differentiable verifier emitting structured per-element feedback, paired with a 50-line force integrator, is a strictly stronger constraint primitive than classifier guidance, RePaint inpainting, or heavy 5000-step legalizers on ISPD2005. An intra-sampling variant gives a useful Pareto knob on the largest circuits. We hope our framing of structured verifier feedback as an interface between diffusion samplers and discrete predicates will generalize beyond placement. All experiments use publicly released data and checkpoints; seeds ({42,123,300,2024}) and code are in the supplementary, with all reported numbers reproducible from released JSONs. We see no negative societal impact specific to this contribution beyond existing concerns about automating physical design.

References

[1] Arpit Bansal, Hong-Min Chu, Avi Schwarzschild, Soumyadip Sengupta, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Universal guidance for diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023. arXiv:2302.07121.

[2] Tung-Chieh Chen, Zhe-Wei Jiang, Tien-Chang Hsu, Hsin-Chen Chen, and Yao-Wen Chang. NTUplace3: An analytical placer for large-scale mixed-size designs with preplaced blocks and density constraints. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 27(7):1228–1240, 2008.

[3] Chung-Kuan Cheng, Andrew B. Kahng, Ilgweon Kang, and Lutong Wang. RePlAce: Advancing solution quality and routability validation in global placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, 38(9):1717–1730, 2019.

[4] Prafulla Dhariwal and Alex Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[5] Yao Lai, Jinxin Liu, Zhentao Tang, Bin Wang, Jianye Hao, and Ping Luo. ChiPFormer: Transferable chip placement via offline decision transformer. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2306.14744.

[6] Vinit Sunder Lee, Chung-Kuan Cheng, and Bo-Yuan Yang. Chip placement with diffusion models. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2407.12282.

[7] Yibo Lin, Shounak Dhar, Wuxi Li, Haoxing Ren, Brucek Khailany, and David Z. Pan. DREAMPlace: Deep learning toolkit-enabled gpu acceleration for modern VLSI placement. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*, 2019.

[8] Andreas Lugmayr, Martin Danelljan, Andres Romero, Fisher Yu, Radu Timofte, and Luc Van Gool. RePaint: Inpainting using denoising diffusion probabilistic models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

A Implementation details of the verifier

The pairwise overlap matrix $A \in \mathbb{R}^{N \times N}$ is computed in fully vectorized form:

```
cx, cy = centers[:, 0], centers[:, 1]
sw, sh = sizes[:, 0], sizes[:, 1]
ovx = clamp_min(min(cx[:,None]+sw[:,None]/2, cx[None,:]+sw[None,:]/2)
              - max(cx[:,None]-sw[:,None]/2, cx[None,:]-sw[None,:]/2), 0)
ovy = clamp_min(min(cy[:,None]+sh[:,None]/2, cy[None,:]+sh[None,:]/2)
              - max(cy[:,None]-sh[:,None]/2, cy[None,:]-sh[None,:]/2), 0)
A = ovx * ovy;  A.fill_diagonal_(0)
```

Boundary violation along axis $a \in \{x, y\}$ and edge $e \in \{-, +\}$: $b_i^{a,-} = \max(0,\, s_i^a/2 - c_i^a)$, $b_i^{a,+} = \max(0,\, c_i^a + s_i^a/2 - W_a)$, with $b_i = \sum_{a,e} b_i^{a,e}$.

B Lambda sweep: per-circuit detail

The cross-circuit summary appears in the main paper. Table 6 gives the per-circuit, per-λ Pareto numbers behind that summary ($n = 3$ seeds per cell, mean over seeds). Note the strict-Pareto rate per operating point: from $0/18$ at $\lambda = 0$ (pure repulsion, HPWL inflates $\sim 162\%$) to $16/18$ at $\lambda = 4$.

	λ=0		λ=0.25		λ=0.5		λ=1		λ=2		λ=4	
Circuit	Δv	Δh	Δv	Δh	Δv	Δh	Δv	Δh	Δv	Δh	Δv	Δh
adaptec1	-41.7	$+226.2$	-39.5	$+87.1$	-40.1	$+47.5$	-36.4	$+31.1$	-37.0	-7.3	-34.5	-3.7
adaptec2	-55.5	$+251.0$	-48.2	$+128.1$	-45.8	$+85.4$	-44.6	$+50.3$	-46.6	$+16.7$	-39.9	-19.1
adaptec3	-51.0	$+124.0$	-50.6	$+75.9$	-50.9	$+56.4$	-51.0	$+34.5$	-52.9	$+17.5$	-55.2	$+3.4$
adaptec4	-62.1	$+104.3$	-60.9	$+62.7$	-60.6	$+36.3$	-60.8	$+6.9$	-58.4	-11.4	-56.8	-25.5
bigblue1	-20.0	$+142.1$	-18.7	-22.3	-22.9	-61.2	-23.4	-77.0	-25.5	-73.5	-22.6	-75.9
bigblue3	-78.6	$+122.0$	-78.4	$+72.3$	-79.0	$+54.5$	-79.2	$+37.4$	-79.3	$+19.5$	-78.5	-3.9
strict-P	$0/18$		$2/18$		$3/18$		$4/18$		$8/18$		$\mathbf{16/18}$	

Table 6: Per-circuit $\Delta v\,\%$ and $\Delta h\,\%$ across the λ sweep, mean over 3 seeds. Strict-Pareto count rises monotonically with λ, peaking at 89% ($16/18$) at λ=4.

C Component ablation: full table and notes

Table 7 reproduces Table 6 from §4.8 with all 18 (circuit, seed) trials. Two implementation notes: (i) "− overlap signal" and "− repulsive force" produce identical numbers because the repulsive force is the only consumer of the pairwise overlap matrix; (ii) the "− boundary signal" variant is incidentally redundant on these saturated drafts because the repulsive force pushes boundary-violating macros incidentally back inside.

variant	$\Delta v\,\%$	$\Delta h\,\%$	strict-Pareto	time (s)
full (control)	$-50.1_{\pm 17.1}$	$-5.1_{\pm 34.8}$	9/18	1.89
severity soft-mask	$-21.8_{\pm 16.9}$	$+43.7_{\pm 23.5}$	1/18	1.55
− overlap signal	$+79.2_{\pm 57.4}$	$-85.0_{\pm 7.2}$	0/18	1.43
− boundary signal	$-82.9_{\pm 4.1}$	$-8.9_{\pm 30.8}$	9/18	1.72
− attractive force	$-51.4_{\pm 18.5}$	$+138.4_{\pm 53.6}$	0/18	1.44
− repulsive force	$+79.2_{\pm 57.4}$	$-85.0_{\pm 7.2}$	0/18	1.46
random selector	$-50.1_{\pm 17.1}$	$-5.1_{\pm 34.8}$	9/18	1.58
uniform (all macros)	$-50.1_{\pm 17.1}$	$-5.1_{\pm 34.8}$	9/18	1.49

Table 7: Component ablation, mean ± std over 18 (circuit, seed) pairs. Raw data: `results/vsr_extra/component_ablation.json`.

D Failure-mode analysis: full list

Of the 24 trials at $\lambda = 2$, 14 are flagged as not strict-Pareto improving the raw draft (Table 8). Failures cluster on adaptec3 and bigblue3, both of which have hub macros where moving any single macro drags long edge tails.

Circuit	seed	$\Delta v\,\%$	$\Delta h\,\%$	failure flags
adaptec1	123	-37.5	$+42.2$	HPWL$>+20\%$, non-Pareto
adaptec2	123	-60.5	$+6.4$	non-Pareto
adaptec2	300	-48.2	$+25.7$	HPWL$>+20\%$, non-Pareto
adaptec2	2024	-61.8	$+5.0$	non-Pareto
adaptec3	42	-57.9	$+23.1$	HPWL$>+20\%$, non-Pareto
adaptec3	123	-52.9	$+26.9$	HPWL$>+20\%$, non-Pareto
adaptec3	300	-51.8	$+33.9$	HPWL$>+20\%$, non-Pareto
adaptec3	2024	-51.2	$+35.3$	HPWL$>+20\%$, non-Pareto
bigblue1	123	-13.9	-80.8	weak-violation-cut
bigblue1	2024	-20.0	-47.8	weak-violation-cut
bigblue3	42	-77.7	$+15.0$	non-Pareto
bigblue3	123	-81.5	$+17.8$	non-Pareto
bigblue3	300	-72.8	$+14.2$	non-Pareto
bigblue3	2024	-77.1	$+12.9$	non-Pareto

Table 8: All 14 trials flagged at λ=2. Raw data: `results/vsr_extra/failure_cases.csv`.

E Toy cross-domain experiment

We trained a tiny ($\sim$ 200 K-parameter) attention-GNN-style denoiser on 1000 synthetic legal 30-disk layouts (canvas $[0,1]^2$, radii $\in [0.018, 0.035]$, sparse random graph with 60 edges) for 30,000 training steps. At inference, 50 test prompts $\times$ 5 seeds $=$ 250 runs per method (Table 9).

Method	violation count	overlap area	wirelength
baseline (raw)	11.2	0.246	42.4
classifier guidance	27.8	**0.000**	227.9 (5.4$\times$ baseline)
RePaint, binary mask	**3.4**	0.064	38.6
VSR-post (ours)	6.2	**0.030**	41.0
VSR-intra (soft)	8.3	0.149	42.3

Table 9: Toy 2D non-overlap, mean over 250 runs/method. Classifier guidance kills overlap but inflates wirelength 5.4$\times$, mirroring the placement-domain finding; verifier-guided structured repair preserves both. Raw rows: `results/toy/toy_results.json`.

F Neural repair operator: full training details

The negative-result GNN repair operator referenced in §5 spans six training regimes. Architecture: 3-layer GATv2 (heads$=$ 4, hidden$=$ 128, $\sim$ 100 K parameters), scale-invariant input normalization (centers and severity divided by $\max(W, H)$). Node features: $[\text{severity}, \text{boundary}, \text{n_overlappers}, s^w, s^h]$. Edge features: 4-dim pin offsets from `edge_attr`. L2 loss on predicted displacement.

Variants. (1) *Synthetic teacher*: hand-crafted operator outputs on synthetic graphs (2500 train, 200 val). (2) *Real teacher*: hand-crafted on diffusion-sampled ISPD2005 placements (3 samples $\times$ up to 3 circuits = 3–9 samples). (3) *Data augmentation*: per-sample $\times$50 random perturbations. (4) *Trajectory distillation*: (x_k, x_{k+1}) pairs from every hand-crafted step; 30 steps $\times$ 3 samples $=$ 90 pairs. (5) *Ground-truth target*: ISPD2005 industrial `.pl` placements. (6) *Residual learning*: predict $x_{GT} - x_{\text{hand}}$.

Findings. Best validation L2 was 0.0037 (trajectory distillation), but at inference on ISPD2005 the best variant (residual s=0.1) improved violations on 1 of 6 circuits while degrading HPWL on every

other circuit. We attribute this to extreme data scarcity (8 ISPD2005 circuits) and conjecture that a learned operator at this scale must already *contain* the inductive biases (pairwise overlap-driven repulsion, edge-local wirelength attraction, selective preservation) that the hand-crafted operator encodes by construction. Detailed per-circuit evaluations: `results/neural_vsr/eval_*.json`.

G Convergence curves and step-size ablation

Figure 6 shows per-iteration (v_t, h_t) trajectories for $T = 300$ iterations on adaptec1 and bigblue1 at $\lambda = 2$. Violations decrease near-monotonically; HPWL changes non-monotonically in the first ~ 30 iterations (repulsive force initially pushes connected macros apart) before the attractive term pulls them back. Plateau is reached by $T \approx 100$, confirming the default budget.

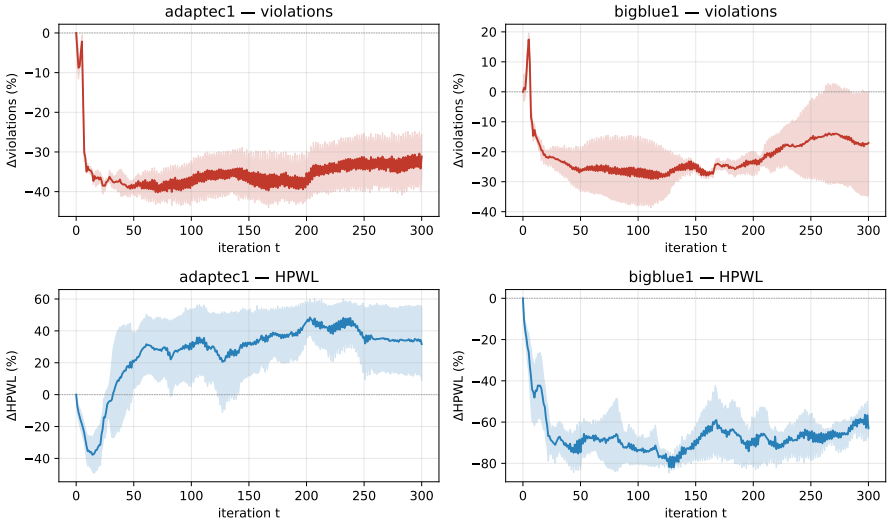

Figure 6: Per-iteration convergence on adaptec1 and bigblue1, 3 seeds. Top: violations. Bottom: HPWL. Shaded band: seed std.

Step-size ablation: $\eta = 0.3$ (default) is the robust middle ground; $\eta = 1.0$ gains marginal violation reduction at higher HPWL variance. Step-budget ablation: violations plateau by $T = 100$ on every circuit; $T = 500$ adds < 2 pp at $5\times$ wall-clock.

H Reproducibility and released artifacts

All code, configs, and result JSONs are released. The main-paper numbers reproduce on a single A800 80 GB GPU (full sweep ~ 70 min):

```
python scripts/run_main_neurips.py        # main result, ~30 min
python scripts/run_timestep_sweep.py      # timestep sweep, ~12 min
python scripts/run_mem_profile.py         # memory profile, ~16 min
python scripts/run_component_ablation.py  # component ablation, ~12 min
python scripts/compute_neurips_stats.py
python scripts/analyze_extra_experiments.py
python scripts/analyze_component_ablation.py
```

Seeds: {42, 123, 300, 2024} for 4-seed runs; {42, 123, 300} for 3-seed sweeps. Result JSONs: `results/ispd2005/` and `results/vsr_extra/`.